

05/05/2026

INGENERIA EN SISTEMAS

UNIVERSIDAD MARIANO GALVEZ, BOCA DEL MONTE, GUATEMALA, GUATEMALA

Ingeniero; Melvin Cali

Actividad:



Integrantes:

- Emanuel Feliciano Orozco Cifuentes
7690-23-6103
- Angel Steve Morejón Martínez
7690-23-21973
- Angel Aurelio Tzi Cucul
7690-23-3711

Guatemala, 18 mayo de 2026

Introducción

El presente describe la arquitectura, diseño e implementación del sistema de gestión de vehículos basado en una matriz ortogonal desarrollado en Java. El propósito de este documento es proporcionar una explicación detallada de las clases, métodos, estructuras de datos y lógica utilizada durante el desarrollo del proyecto.

La aplicación implementa una estructura dinámica bidimensional mediante nodos enlazados horizontal y verticalmente, permitiendo la administración eficiente de vehículos dentro de una matriz dispersa. Para ello, se aplicaron conceptos fundamentales de Programación Orientada a Objetos, listas enlazadas y recorridos dinámicos.

Además, el documento explica las responsabilidades de cada clase, el funcionamiento de los métodos principales, las validaciones implementadas y el comportamiento interno de la estructura ortogonal, facilitando el mantenimiento, comprensión y futura expansión del sistema.

Objetivo General

Implementar una estructura de datos dinámica tipo matriz ortogonal utilizando Java y Programación Orientada a Objetos, permitiendo el almacenamiento eficiente y administración bidimensional de vehículos mediante nodos enlazados.

Objetivos Específicos

- Aplicar conceptos de Programación Orientada a Objetos en el desarrollo del sistema.
- Implementar una estructura ortogonal mediante enlaces horizontales y verticales.
- Desarrollar métodos de inserción, búsqueda, eliminación y recorrido de nodos.
- Optimizar el uso de memoria utilizando estructuras dinámicas.
- Diseñar clases modulares y organizadas para facilitar el mantenimiento del sistema.
- Implementar validaciones para evitar registros duplicados o datos inválidos.
- Comprender el funcionamiento práctico de matrices dispersas y estructuras bidimensionales dinámicas.

Arquitectura del proyecto

Clase	Función
Main	Control del menú y validaciones
MatrizVehiculos	Manejo de la matriz ortogonal
VehiculoNodo	Representación de cada vehículo
NodoCabecera	Cabeceras de filas y columnas

Estructura Ortogonal

La matriz ortogonal es una estructura de datos bidimensional dinámica.

Cada nodo puede conectarse:

- Horizontalmente
- Verticalmente

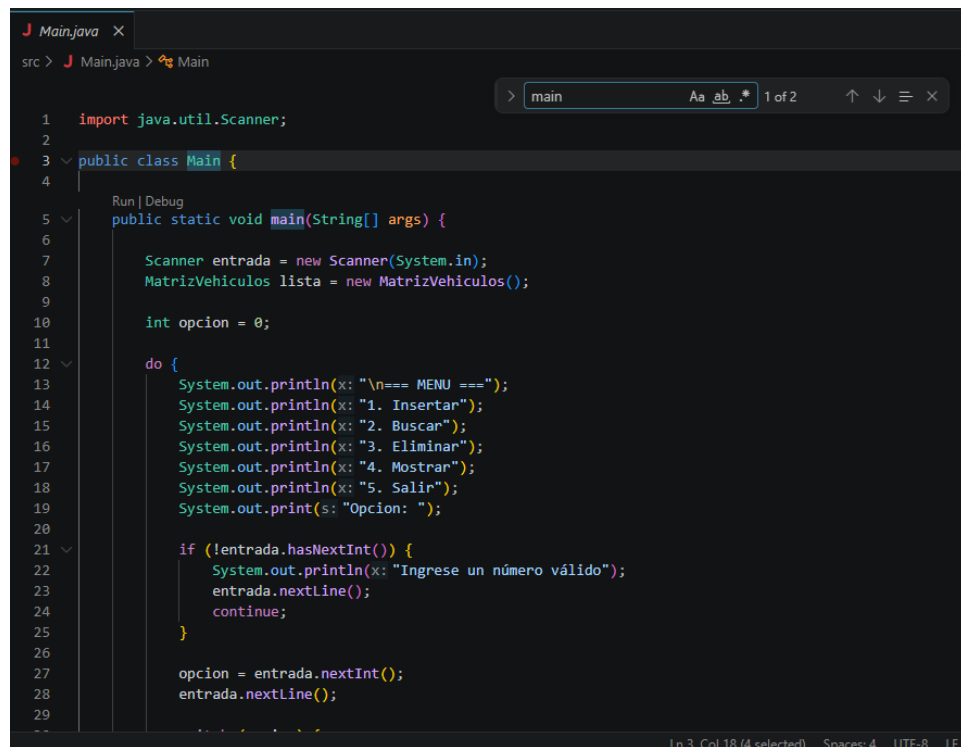
Esto permite recorrer la matriz tanto por filas como por columnas.

CLASES

Clase main:

La clase Main controla, responsabilidades:

- El menú principal
- Entrada de datos
- Validaciones
- Comunicación con la matriz



```
1  import java.util.Scanner;
2
3  public class Main {
4
5      public static void main(String[] args) {
6
7          Scanner entrada = new Scanner(System.in);
8          MatrizVehiculos lista = new MatrizVehiculos();
9
10         int opcion = 0;
11
12         do {
13             System.out.println(x: "\n=== MENU ===");
14             System.out.println(x: "1. Insertar");
15             System.out.println(x: "2. Buscar");
16             System.out.println(x: "3. Eliminar");
17             System.out.println(x: "4. Mostrar");
18             System.out.println(x: "5. Salir");
19             System.out.print(s: "Opcion: ");
20
21             if (!entrada.hasNextInt()) {
22                 System.out.println(x: "Ingrese un número válido");
23                 entrada.nextLine();
24                 continue;
25             }
26
27             opcion = entrada.nextInt();
28             entrada.nextLine();
29
30         } while (opcion != 5);
31     }
32 }
```

Método main()

Función; Controla el flujo principal del programa.

Funciones realizadas

- Muestra el menú
- Lee opciones
- Ejecuta operaciones
- Valida entradas del usuario

Metodo validaciónPlaca()

Función

Verifica que la placa ingresada tenga un formato correcto.

Validaciones realizadas

- No permite valores nulos
- Convierte a mayúsculas
- Verifica longitud
- Verifica que inicie con letra
- Verifica presencia de números
- No permite símbolos

```
11
12 // VALIDACIONES PRIMERO
13 if (existePlaca(placa)) {
14     System.out.println(x: "Placa duplicada");
15     return;
16 }
17
18 if (existePosicion(fila, columna)) {
19     System.out.println(x: "Ya existe un vehículo en esa posición");
20     return;
21 }
22
23 NodoCabecera filaCab = obtenerCabeceraFila(fila);
24 NodoCabecera colCab = obtenerCabeceraColumna(columna);
25
26 VehiculoNodo nuevo = new VehiculoNodo(fila, columna, placa, color, linea, modelo, propietario);
27
28 // ♦ INSERTAR EN FILA (horizontal)
29 if (filaCab.acceso == null) {
30     filaCab.acceso = nuevo;
31 } else {
32     VehiculoNodo actual = filaCab.acceso;
33     VehiculoNodo anterior = null;
34
35     while (actual != null && actual.getColumna() < columna) {
36         anterior = actual;
37         actual = actual.getSiguiente();
38     }
39
40     if (anterior != null) {
41         anterior.setSiguiente(nuevo);
42     } else {
43         filaCab.acceso = nuevo;
44     }
45 }
46
47 return filaCab;
48 }
```

Clase MatrizVehiculos

Es la clase principal de la estructura ortogonal, administra:

- Inserciones
- Búsquedas
- Eliminaciones
- Recorridos
- Validaciones internas

Variables principales:

```
1 public class MatrizVehiculos {  
2  
3     private NodoCabecera filas;  
4     private NodoCabecera columnas;  
5  
6 }
```

Función

Almacenan las cabeceras de filas y columnas de la matriz.

Método Insertar ()

```
5  
6  
7     // ♦ INSERTAR POR POSICIÓN REAL  
8     public void insertar(int fila, int columna,  
9         String placa, String color,  
10        String linea, String modelo, String propietario) {  
11 }
```

Función

Inserta un nuevo vehículo dentro de la matriz ortogonal.

Procesos

1. Verifica placas duplicadas
2. Verifica posiciones ocupadas
3. Obtiene cabeceras
4. Crea el nodo
5. Inserta horizontalmente
6. Inserta verticalmente

Inserción Horizontal

La inserción horizontal enlaza nodos por columnas;

```
34
35     while (actual != null && actual.getColumna() < columna) {
36         anterior = actual;
37         actual = actual.getDerecha();
38     }
39
```

Permite recorrer la fila de izquierda a derecha.

Inserción Vertical

La inserción vertical enlaza nodos por filas.

```
55
56     while (actual != null && actual.getFila() < fila) {
57         anterior = actual;
58         actual = actual.getAbajo();
59     }
```

Permite recorrer la columna de arriba hacia abajo.

Método buscar()

```
137 public void buscar(String valor) {
138
139     NodoCabecera fila = filas;
140     boolean encontrado = false;
141
142     while (fila != null) {
143
144         VehiculoNodo actual = fila.acceso;
145
146         while (actual != null) {
147
148             if (actual.coincideCon(valor)) {
149                 System.out.println(actual);
150                 encontrado = true;
151             }
152
153             actual = actual.getDerecha();
154         }
155
156         fila = fila.siguiente;
157     }
158 }
```

Función

Busca coincidencias utilizando cualquier atributo del vehículo.

Atributos comparados

- Placa
- Color
- Línea
- Modelo
- Propietario

Recorrido

El recorrido se realiza fila por fila.

Método eliminar()

```
163 public void eliminar(String placa) {  
164  
165     NodoCabecera fila = filas;  
166  
167     while (fila != null) {  
168  
169         VehiculoNodo actual = fila.acceso;  
170         VehiculoNodo anterior = null;  
171  
172         while (actual != null) {  
173  
174             if (actual.esPlaca(placa)) {  
175  
176                 // ♦ eliminar en fila  
177                 if (anterior == null) {  
178                     fila.acceso = actual.getDerecha();  
179                 } else {  
180                     anterior.setDerecha(actual.getDerecha());  
181                 }  
182             }  
183             anterior = actual;  
184             actual = actual.getDerecha();  
185         }  
186         fila = fila.getDerecha();  
187     }  
188 }
```

Función

Elimina un vehículo utilizando la placa como identificador.

Proceso

1. Busca el nodo
2. Desconecta horizontalmente
3. Desconecta verticalmente
4. Actualiza referencias

public void mostrar()

```
222 public void mostrar() {
223
224     NodoCabecera fila = filas;
225
226     while (fila != null) {
227
228         VehiculoNodo actual = fila.acceso;
229
230         while (actual != null) {
231             System.out.print(actual + " ");
232             actual = actual.getDerecha();
233         }
234
235         System.out.println();
236         fila = fila.siguiente;
237     }
238 }
```

Función

Recorre toda la matriz mostrando los vehículos almacenados.

Método existePlaca()

```
239 private boolean existePlaca(String placa) {
240
241     NodoCabecera fila = filas;
242
243     while (fila != null) {
244
245         VehiculoNodo actual = fila.acceso;
246
247         while (actual != null) {
248             if (actual.esPlaca(placa)) return true;
249             actual = actual.getDerecha();
250         }
251
252         fila = fila.siguiente;
253     }
254
255     return false;
256 }
```

Función

Verifica si una placa ya se encuentra almacenada.

Método existePosicion()

```
256 }
257 private boolean existePosicion(int fila, int columna) {
258
259     NodoCabecera filaCab = filas;
260
261     while (filaCab != null) {
262
263         if (filaCab.indice == fila) {
264
265             VehiculoNode actual = filaCab.acceso;
266
267             while (actual != null) {
268                 if (actual.getColumna() == columna) {
269                     return true;
270                 }
271                 actual = actual.getDerecha();
272             }
273
274             filaCab = filaCab.siguiente;
275         }
276     }
277
278     return false;
279 }
280 }
```

Función

Verifica si ya existe un vehículo en una posición específica.

Clase VehiculoNodo

```
1 public class VehiculoNodo {  
2
```

Representa cada vehículo dentro de la matriz.

Cada nodo almacena:

- Información del vehículo
- Posición
- Referencias de enlaces

Estructura del Nodo

```
5  
6 private VehiculoNodo derecha;  
7 private VehiculoNodo abajo;  
8
```

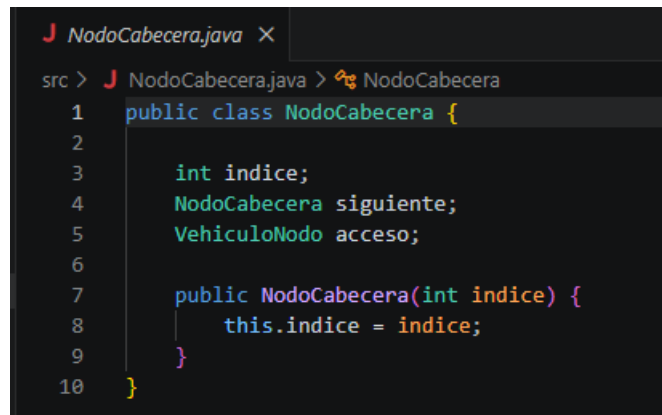
Explicación

- derecha conecta nodos de la misma fila
- abajo conecta nodos de la misma columna

Métodos Importantes de VehiculoNodo

Método	Función
getDerecha()	Obtiene nodo derecho
setDerecha()	Asigna nodo derecho
getAbajo()	Obtiene nodo inferior
setAbajo()	Asigna nodo inferior
coincideCon()	Verifica coincidencias
esPlaca()	Compara placas

Clase NodoCabecera



```
J NodoCabecera.java X
src > J NodoCabecera.java > NodoCabecera
1 public class NodoCabecera {
2
3     int indice;
4     NodoCabecera siguiente;
5     VehiculoNodo acceso;
6
7     public NodoCabecera(int indice) {
8         this.indice = indice;
9     }
10 }
```

Representa las cabeceras de filas y columnas.

Cada cabecera almacena:

- Índice de fila o columna
- Referencia al primer nodo
- Referencia al siguiente encabezado

Ventajas de la Matriz Ortogonal

- Uso eficiente de memoria
- Inserciones dinámicas
- Recorridos bidimensionales
- No requiere tamaño fijo
- Ideal para matrices dispersas

Desventajas

- Mayor complejidad
- Recorridos más lentos que un arreglo
- Mayor cantidad de referencias

Conclusiones

La implementación del sistema permitió aplicar de manera práctica conceptos avanzados de estructuras de datos y Programación Orientada a Objetos utilizando Java. El uso de una matriz ortogonal demostró ser una solución eficiente para el almacenamiento dinámico de información bidimensional, optimizando memoria y facilitando recorridos tanto horizontales como verticales.

El proyecto logró integrar correctamente funcionalidades de inserción, búsqueda, eliminación y visualización de datos mediante nodos enlazados, manteniendo la integridad de la estructura durante cada operación realizada.

Asimismo, el desarrollo del sistema fortaleció conocimientos relacionados con estructuras dinámicas, manipulación de referencias, validaciones y organización modular del código, proporcionando una base sólida para futuros proyectos más complejos relacionados con gestión de información y estructuras avanzadas de datos.